

MedDetect AI (v2.0.0) - Comprehensive Technical Report

MedDetect AI is a state-of-the-art, AI-powered multi-disease detection platform designed to assist users in identifying potential health conditions based on symptoms. It bridges machine learning diagnostics, generative AI interactions, and telehealth capabilities to form a complete patient and doctor ecosystem.

1. How It Was Implemented (Architecture)

The platform is designed using a modern full-stack approach, utilizing **Streamlit** as both the backend logic orchestrator and the frontend rendering engine.

1.1 UI & Frontend Design

- **Premium Glassmorphism:** Custom CSS (`shared_ui.py`) is injected directly into Streamlit to create a sleek, frosted-glass interface that feels like a native mobile health app.
- **Injected JS Components:** A floating AI chatbot component ("Dr. Docyote") is manually injected into the DOM. This bypasses standard Streamlit limitations by using a hidden form bridge to achieve instantaneous, dynamic UI updates and a persistent presence across pages.

1.2 The AI & Machine Learning Pipeline

The intelligence is split into two specialized engines:

1. **Generative NLP Pipeline:** Uses **Groq (Whisper)** for speech-to-text rendering in milliseconds, and **Google Gemini 2.0 Flash** for translating natural layman descriptions (and multiple languages like Hindi) into strict, clinical symptoms.
2. **Machine Learning Classifier:** A **Random Forest Classifier** (`scikit-learn`) trained on 9,400+ rows covering 221 unique diseases parses the exact symptom vector to output a probabilistic prediction of the illness.

1.3 Telehealth & Integration

- Once a diagnosis is reached, the system uses **FPDF** to render a high-quality PDF report and **smtplib** to natively email it to the patient.
 - It leverages the **Jitsi Meet API** to generate instant consultation rooms for urgent diagnoses.
-

2. Database Structure

MedDetect AI relies on **Supabase (PostgreSQL)** to handle authentication and relational data storage. The database schema focuses on 4 core tables:

2.1 `predictions` Table

Stores a user's diagnostic history.

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → Auth)
- `symptoms` (JSON/Array): The mapped clinical symptoms experienced.
- `predicted_disease` (String): The ML model's highest-confidence output.
- `confidence` (Float): Percentage probability from the model.

- `created_at` (Timestamp)

2.2 doctors Table

Physician profiles for the telehealth booking workflow.

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → Auth)
- `name` , `email` , `specialty` , `phone` (Strings)
- `city` , `hospital_name` , `hospital_address` (Strings)
- `created_at` (Timestamp)

2.3 appointments Table

Coordinates video consultations between patients and doctors.

- `id` (UUID, Primary Key)
- `patient_id` / `patient_email` (Foreign Key / String)
- `doctor_id` / `doctor_name` / `specialty` (Foreign Keys / Strings)
- `appointment_date` / `appointment_time` (Date/Time)
- `meeting_url` (String): Secure Jitsi Meet generated link.
- `status` (Enum: `upcoming` , `completed` , `cancelled`)
- `created_at` (Timestamp)

2.4 prescriptions Table

Digital records issued by doctors post-consultation.

- `id` (UUID, Primary Key)
- `appointment_id` (UUID, Foreign Key → appointments)
- `doctor_id` / `patient_id` / `patient_email` (Foreign Keys / Strings)
- `diagnosis` (String): Doctor's verified diagnosis.
- `medicines` (JSON/Array): List of prescribed drugs and dosages.
- `notes` (Text): Additional lifestyle advice.
- `created_at` (Timestamp)

3. Python Modules Used and Why

The backend logic is modularized effectively within the `modules/` directory. Breaking the system into 12 core modules ensures scalability and ease of debugging.

1. **database.py** : A robust wrapper for the Supabase API. Contains `@with_retry()` decorators to handle cloud network glitches gracefully without crashing the app.
2. **disease_info.py** : A localized static database of medical mappings (e.g., matching a predicted disease to the correct medical specialty, offering precautions, and suggesting remedies).
3. **doctor_locator.py** : Invokes the **Groq Llama-3** LLM to contextually figure out exactly which type of specialist a patient should see based on their geolocation and disease.
4. **email_service.py** : A threading-enabled SMTP client. Used to send offline PDF reports and appointment confirmations without hanging the main Streamlit thread.
5. **floating_chat.py** : Crafts the "Dr. Docyote" custom UI component. It converts raw HTML/JS/CSS templates and establishes the hidden two-way session bridge so the chat doesn't reload the main app interface.

6. `logger.py` : Implements standardized console logging to track application state and catch exceptions cleanly during runtime.
 7. `medicine_db.py` : Holds detailed pharmacological data. Required for the Doctor Portal's prescription auto-complete functionality to prevent medication spelling errors.
 8. `ml_engine.py` : Abstractly loads the binary `.pk1` Random Forest model using `joblib` and exposes an inference method to convert raw text symptoms into ML outputs.
 9. `nlp_extractor.py` : The "Translation Engine." It communicates with Google Gemini to filter out noise, handle negative sentiment ("I do not have a cough"), and strictly match user speech to the model's 382 exact token features.
 10. `pdf_generator.py` : Uses `fpdf2` to construct branded, multi-page diagnostic PDFs that summarize the patient's symptoms, diagnosis history, and machine learning results.
 11. `prescription_pdf.py` : A specialized sub-library of `pdf_generator` tailored exclusively for official prescription generation (including signature blocks and Rx headers).
 12. `shared_ui.py` : Standardizes visual elements across pages (like sidebars and CSS variables). Used to enforce UI consistency so components don't have to redefine styling rules.
-

4. How the Code Works (Data Flow)

The application follows a strictly linear, stateful execution loop managed by Streamlit:

1. **Initialization (`app.py`)** : The application boots, loads the ML environment, establishes the database connection, and injects `shared_ui.py` elements (like the CSS theme and sidebar).
2. **Authentication Flow (`0_🔑_Sign_In.py`)** : Session state relies on `supabase.auth` . No sensitive data can be processed until a session token has been verified and stored in `st.session_state` .
3. **Extraction & Inference (`1_🔍_Symptom_Checker.py`)** :
 - The user inputs data (voice or text).
 - This data is sent through `nlp_extractor.py` . Gemini transforms "My head is pounding" into `['headache']` .
 - The cleansed symptoms array is given to `ml_engine.py` , which immediately predicts the target class (e.g., `Migraine`).
 - The application checks `disease_info.py` to evaluate if the illness severity is Critical, Moderate, or Low.
4. **Response & Action Generation** :
 - The system displays the prediction back to the user on the grid.
 - Concurrently, `pdf_generator.py` compiles the result into a clean document, and `email_service.py` securely mails it to the user's registered inbox.
 - `database.py` commits the outcome to the `predictions` table so it populates the user's `2_📅_Report_History.py` timeline.
5. **Follow-Up (`3_👤_Book_Appointment.py`)** :
 - Depending on the disease, `doctor_locator.py` recommends a specialist.
 - The user can select a registered doctor, triggering `database.py` to save an appointment tuple, and an email confirmation fires off containing an instant video hyperlink (via Jitsi).

This modular structure ensures that the computationally heavy ML steps are safely abstracted away from the UI state, allowing a seamless, lightning-fast patient experience.